

Documentazione Progetto di Reti Informatiche A.A. 2025/2026

Daniele Pisani

1 Scelte implementative riguardo la comunicazione

La comunicazione fra lavagna e utenti avviene tramite protocollo TCP su 127.0.0.1, con la lavagna che si posiziona sulla porta 5678 e gli utenti sulle porte successive, dalla 5679 in poi. Inoltre utilizzo TCP anche per la parte P2P, per la quale ogni utente inizializza un server P2P da usare per comunicare con gli altri utenti. La scelta di usare TCP deriva dal fatto che l'utilizzo di UDP avrebbe aggiunto notevole complessità al progetto. Avrebbe infatti richiesto di gestire gli errori di consegna dei pacchetti e, specialmente nel P2P, sarebbe stato necessario introdurre meccanismi per evitare che la perdita di un messaggio `CHOOSE_USER` bloccasse la selezione del vincitore della card, quali per esempio l'introduzione di un quorum o di una gestione delle ritrasmissioni.

Le connessioni multiple vengono gestite dalla lavagna tramite multiplexing usando `select()`, mentre il processo utente usa un approccio misto. Il thread principale gestisce la connessione con la lavagna e l'input da terminale, un secondo thread ascolta i messaggi P2P dagli altri utenti e un worker separato simula l'esecuzione della card vinta. Questa scelta isola meglio i compiti del processo utente e permette di rispondere ai comandi ricevuti da lavagna o terminale e di ricevere i costi P2P in qualsiasi momento, anche mentre si attende l'elaborazione di una carta.

Per scambiare messaggi viene usato un formato binario standard per costituire l'header, composto da: 32 bit per il tipo del messaggio e 32 bit per la lunghezza del payload, entrambi in network byte order. Molti messaggi, come `PING_USER`, `PONG_LAVAGNA` e `QUIT`, non hanno payload. Gli altri messaggi possiedono un payload di dimensioni variabili, che può contenere sia altri byte di campi relativi al messaggio, sia eventuali stringhe terminate da `\0`. Ho adottato questa soluzione in quanto mi sembrava efficiente per costruire e leggere velocemente i pacchetti necessari per il progetto, spesso utilizzati per trasmettere informazioni numeriche, gestendo le stringhe specificamente solo quando richiesto.

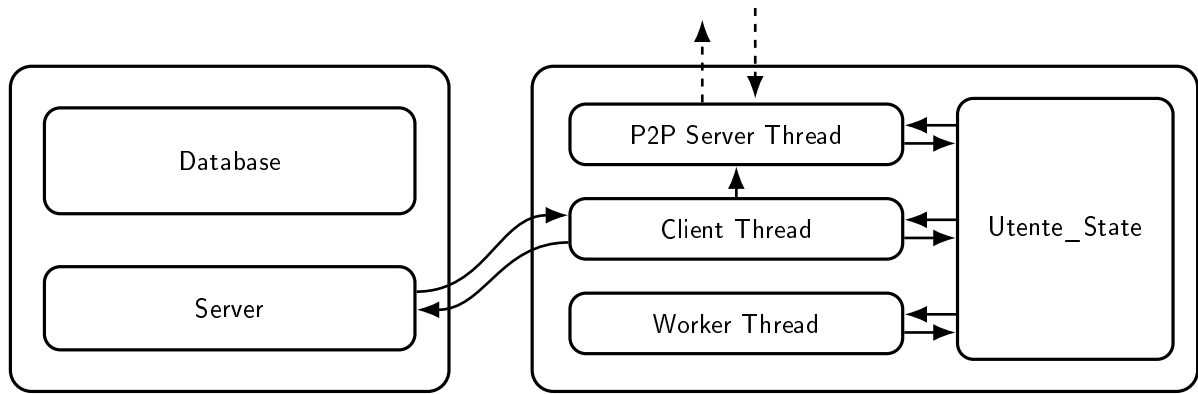
2 Struttura logica del programma

Ho cercato di strutturare il programma in modo modulare. I moduli `server` e `client` astraggono la gestione dei socket, mentre `protocol` contiene la logica comune di serializzazione e deserializzazione dei messaggi. In questo modo sia la lavagna centrale sia il server P2P degli utenti usano la stessa API di basso livello, evitando così di avere duplicati del codice di accettazione, lettura e invio dei messaggi.

La lavagna e l'utente hanno poi due strutture parallele per mantenere lo stato. La lavagna utilizza un `Database` statico con array di utenti e card; gli array hanno dimensione fissa, rispettivamente `MAX_USERS` e `MAX_CARDS`. Non è una struttura ottima né dal punto di vista algoritmico né dal punto di vista della memoria; tuttavia, essendo l'ambito del progetto limitato, ho preferito un'implementazione semplice basata su array che non introducesse troppa complessità nella gestione della memoria. Si potrebbe pensare con future ottimizzazioni di gestire dinamicamente lo spazio utilizzato e ottimizzare le operazioni di estrazione di elementi necessarie al funzionamento del progetto. L'utente invece mantiene una struttura globale `Utente`, incapsulata nel modulo `utente_state.c`: gli altri moduli non vi accedono direttamente, ma devono usare le funzioni esposte dall'API. Inoltre le parti mutabili condivise tra i thread dell'utente sono protette da mutex.

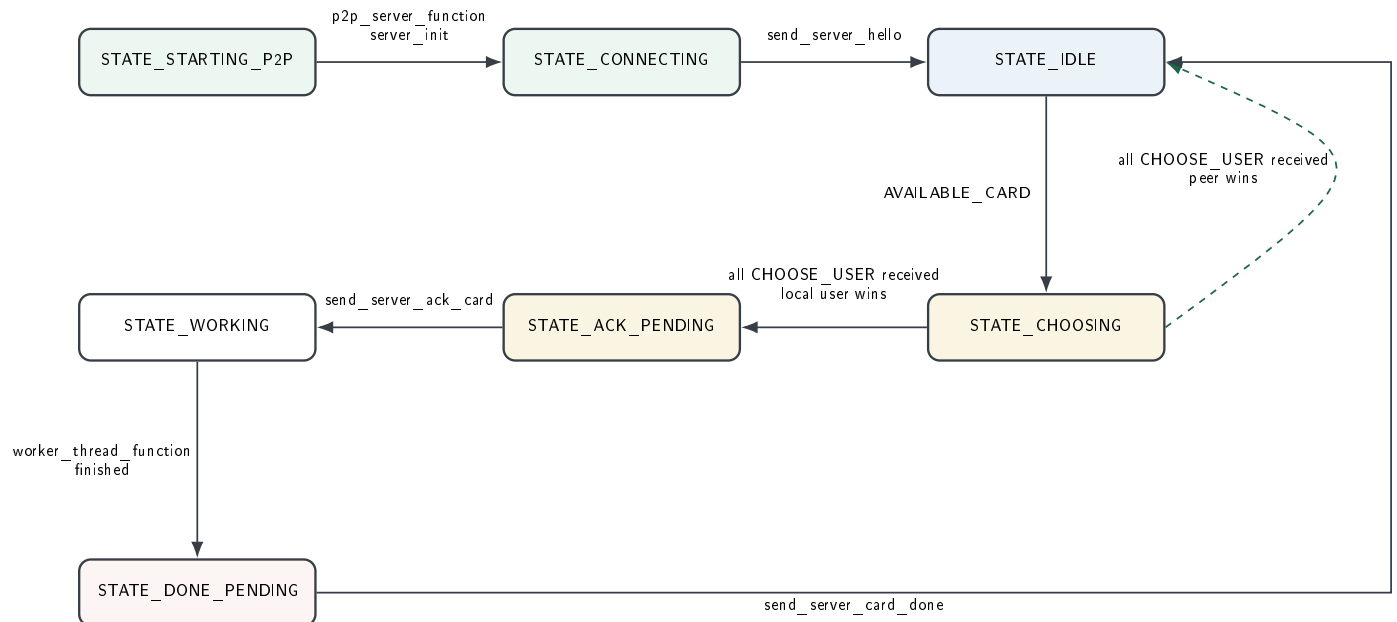
L'utente infine è strutturato per avere due thread oltre al principale, oltre al thread per gestire i messaggi del server P2P utilizza infatti anche un worker thread. Quando un utente vince una card, invia `ACK_CARD` alla lavagna, entra nello stato di lavoro e avvia il worker. Il worker non comunica con l'esterno, si limita a simulare il tempo di esecuzione e a segnalare allo stato condiviso quando la card può essere completata. Il thread principale una volta notificato, cambia lo stato e invia `CARD_DONE` alla lavagna.

Grafico della memoria del progetto



I tre thread del processo utente comunicano attraverso la macchina a stati contenuta in **Utente_State**. Ogni stato descrive quale parte del processo deve agire: in **IDLE** l'utente attende una card, in **CHOOSING** raccoglie i costi e stabilisce se il vincitore è lui o un peer, in **ACK_PENDING** deve confermare la vittoria, in **WORKING** il worker sta lavorando e in **DONE_PENDING** il completamento deve essere notificato alla lavagna.

Grafico degli stati del progetto



La state machine dell'applicazione utente si comporta a grandi linee come descritto in questo grafico. Sono presenti anche due ulteriori stati: **STATE_DISCONNECTING** e **STATE_SHUTTING_DOWN**, usati solo in caso di terminazione o errore per liberare correttamente socket, thread e memoria associata.

3 Compilazione ed esecuzione

Il programma può essere compilato semplicemente con **make**. Il Makefile usa automaticamente **-Wall**, **-Wextra** e **-Wpedantic**, e collega **pthread**. Vengono generati i due eseguibili **lavagna** e **utente**. La lavagna si avvia con **./lavagna**; gli utenti possono essere avviati sia con **./utente <porta>** che senza parametro, in quel caso viene scelta la prima porta libera a partire da 5679. Il comando **make clean** invece rimuove file oggetto ed eseguibili.

La lavagna viene inizialmente popolata con un set di 15 card di esempio e, a patto che il numero di utenti sia sufficiente, l'esecuzione prosegue in autonomia secondo le specifiche. Sui terminali di esecuzione c'è possibile lanciare anche comandi manuali. Dal terminale della lavagna sono accettati: **SHOW_LAVAGNA**, **SHOW_UTENTI**, **SEND_USER_LIST <porta>** e **PING_USER <porta>**. Dal terminale dell'utente sono accettati: **CREATE_CARD <id> <TODO|DOING|DONE> <descrizione>**, **CARD_DONE [id]** e **QUIT**. I parametri indicati tra parentesi quadre sono opzionali, inoltre notare che i comandi **PING_USER** e **CARD_DONE** funzionano correttamente solo rispetto ad un utente in **STATE_WORKING**.